

Documentación de las estructuras utilizadas

Para el desarrollo del proyecto Monopoly Distribuido se implementaron estructuras de datos lineales propias, evitando el uso de colecciones predefinidas de C# para sustituir las estructuras requeridas. Estas estructuras permiten administrar elementos fundamentales del juego, específicamente el tablero, los turnos, las propiedades de cada jugador, las cartas de evento y el historial de transacciones.

3.1 Nodo genérico Nodo<T>

La clase Nodo<T> constituye la unidad básica utilizada para construir las diferentes estructuras enlazadas del proyecto. Se implementó de forma genérica mediante <T>, permitiendo almacenar diferentes tipos de objetos sin necesidad de crear una clase nodo diferente para cada uno.

Cada nodo contiene tres elementos principales:

- Dato: almacena el objeto contenido en el nodo.
- Siguiente: referencia al siguiente nodo.
- Anterior: referencia al nodo anterior.

Su estructura general es:

```
public class Nodo<T>
{
    public T Dato { get; set; }
    public Nodo<T>? Siguiente { get; set; }
    public Nodo<T>? Anterior { get; set; }

    public Nodo(T dato)
    {
        Dato = dato;
        Siguiente = null;
        Anterior = null;
    }
}
```

El uso de genéricos permite utilizar la misma estructura para diferentes elementos del juego. Por ejemplo:

Nodo<Casilla>

Nodo<Jugador>

Nodo<Propiedad>

Nodo<Transaccion>

De esta manera se reutiliza código y se mantiene una implementación uniforme.

3.2 Lista doble circular

La clase ListaDobleCircular se utiliza principalmente para representar **el tablero de Monopoly**.

Se escogió esta estructura porque el tablero es naturalmente circular: después de la última casilla, el jugador debe regresar a la primera. Además, cada nodo posee referencias tanto hacia la siguiente casilla como hacia la anterior.

La relación entre los nodos puede representarse de la siguiente manera:

Salida, pasa a Casilla1, pasa a Casilla2,...,vuelve a Salida.

En la implementación, cuando se agrega el primer nodo:

```
nuevo.Siguiente = nuevo;
```

```
nuevo.Anterior = nuevo;
```

el nodo se referencia a sí mismo. Cuando existen más elementos, el último nodo queda conectado nuevamente con la cabeza de la lista.

La estructura mantiene:

```
public Nodo<Casilla>? Cabeza { get; private set; }
```

```
public int Cantidad { get; private set; }
```

Cabeza permite acceder a la primera casilla del tablero y Cantidad mantiene el número de casillas almacenadas.

Uso dentro del proyecto

El tablero está compuesto por 24 casillas, almacenadas dentro de esta lista. Cada jugador mantiene una referencia a un nodo del tablero:

```
public Nodo<Casilla>? Posicion { get; set; }
```

Para avanzar se utiliza:

```
jugador.Posicion =
```

```
    jugador.Posicion.Siguiente;
```

y para retroceder:

```
jugador.Posicion =
```

```
    jugador.Posicion.Anterior;
```

Esto permite realizar los movimientos sin utilizar índices de arreglos.

La circularidad también permite que, al avanzar desde la última casilla, el jugador llegue automáticamente nuevamente a Salida.

3.3 Cola circular

La estructura ColaCircular<T> fue implementada para manejar elementos que deben repetirse en forma de ciclo.

Mantiene dos referencias principales:

```
private Nodo<T>? frente;
```

```
private Nodo<T>? ultimo;
```

El último nodo se encuentra conectado nuevamente con el frente:

Christian(frente) va a Ulfran va a Isaac va a Fabricio va a frente.

El método:

```
Encolar(T dato)
```

permite agregar elementos a la cola.

El método:

```
ObtenerFrente()
```

permite consultar el elemento que actualmente se encuentra al frente.

Finalmente:

AvanzarTurno()

desplaza las referencias frente y ultimo al siguiente nodo sin eliminar elementos:

frente = frente!.Siguiente;

ultimo = ultimo!.Siguiente;

Uso para los turnos

Esta estructura se utiliza para controlar el orden de los cuatro jugadores:

Christian

...

Ulfran

...

Isaac

...

Fabrizio

...

Christian

...

...

Cuando un jugador termina su turno, la cola avanza al siguiente jugador.

También permite manejar jugadores que pierden un turno o jugadores que han sido eliminados, omitiéndolos cuando corresponda.

Uso para cartas de evento

La misma estructura genérica se utiliza para las cartas:

ColaCircular<CartaEvento>

Después de utilizar una carta, se llama:

cartasEvento.AvanzarTurno();

por lo que la siguiente carta pasa al frente y la utilizada queda nuevamente disponible después de recorrer todas las demás.

Esto permite reutilizar continuamente el conjunto de cartas sin eliminarlas.

3.4 Lista simple genérica

La clase:

ListaSimple<T>

representa una lista enlazada donde cada nodo mantiene únicamente la conexión necesaria para avanzar hacia el siguiente elemento.

Su funcionamiento general es:

Cabeza

...

Nodo 1 pasa a Nodo 2 pasa a Nodo 3 pasa a null

Cuando se agrega un elemento, se crea un nuevo nodo:

Nodo<T> nuevo = new Nodo<T>(dato);

Si la lista está vacía, este se convierte en la cabeza. En caso contrario, se recorre la lista hasta encontrar el último nodo y se agrega el nuevo elemento.

Propiedades de los jugadores

Cada jugador posee:

public ListaSimple<Propiedad> Propiedades

{

get;

private set;

}

Por ejemplo:

Christian

...

Avenida Coto Brus

...

Avenida Vermont

...

Avenida Purral

...

null

Cuando el jugador compra una propiedad, esta se agrega mediante:

jugador.Propiedades.Agregar(propiedad);

La lista también se utiliza para calcular el patrimonio del jugador recorriendo sus propiedades y sumando sus precios de compra al saldo disponible.

3.5 Historial de transacciones

El historial del banco utiliza también una estructura enlazada implementada mediante:

ListaSimple<Transaccion>

Cada vez que ocurre una operación económica se crea un objeto Transaccion y se agrega al historial:

Historial.Agregar(transaccion);

Las transacciones permiten almacenar información como:

ID

Fecha y hora

Número de turno

Tipo

Jugador de origen

Jugador de destino

Monto

Descripción

Por ejemplo:

Transacción 1

...

Compra

Christian va a Banco

¢200

...

Transacción 2

...

Alquiler

Ulfran paga a Christian

¢25

...

Transacción 3

...

Impuesto

Isaac va a Banco

¢200

La estructura permite recorrer las transacciones desde la más antigua hasta la más reciente. También se implementó la consulta en orden inverso, así como búsquedas por jugador y por tipo de transacción.

Finalmente, el historial puede exportarse a un archivo:

transacciones.txt

para mantener un registro externo de las operaciones realizadas durante la partida.

3.6 Resumen de estructuras utilizadas

Estructura	Tipo almacenado	Utilización
Nodo<T>	Genérico	Base para las estructuras enlazadas
ListaDobleCircular	Casilla	Representación del tablero
ColaCircular<Jugador>	Jugador	Administración de turnos
ColaCircular<CartaEvento>	CartaEvento	Rotación de cartas de evento
ListaSimple<Propiedad>	Propiedad	Propiedades pertenecientes a cada jugador
ListaSimple<Transaccion>	Transaccion	Historial de operaciones económicas

3.7 Justificación de las estructuras seleccionadas

Las estructuras fueron seleccionadas según el comportamiento requerido por cada componente del juego. La lista doble circular es adecuada para el tablero debido a que permite desplazamientos en ambas direcciones y conecta la última casilla nuevamente con la primera. La cola circular permite mantener ciclos continuos, por lo que resulta apropiada tanto para el orden de los jugadores como para la reutilización de las cartas de evento.

Por otro lado, la lista simple resulta suficiente para almacenar las propiedades y las transacciones, ya que estos elementos se agregan y posteriormente se recorren principalmente de forma secuencial.

El uso de estructuras genéricas mediante Nodo<T> también permite reutilizar la misma lógica para diferentes tipos de objetos, reduciendo duplicación de código y facilitando el mantenimiento del programa.